

Berliner Hochschule für Technik

Fachbereich VII: Elektrotechnik - Mechatronik - Optometrie

Bachelorarbeit

**Vergleich verschiedener Dateisysteme
für den SD-Karten-Einsatz
in eingebetteten Systemen**

Autor: Michael Kolorz
Matrikelnummer: 100359
Studiengang: Elektrotechnik (B.Eng.)
Schwerpunkt: Kommunikationstechnik
Erstprüfer: Prof. Dr. Peter Gober
Zweitprüfer: Prof. Dr. David Dietrich
Semester: Sommersemester 2026
Abgabedatum: 1. Oktober 2026

Inhaltsverzeichnis

1	Einleitung	2
2	Hardware-Design und Schnittstellen	2
2.1	Einleitung	2
2.2	Idee und Schaltplan	3
2.2.1	Spannungsversorgung	4
2.2.2	USB-C	4
2.2.3	SPI-Interfaces	4
2.2.4	Programmier-Interfaces	5
2.2.5	VREF und ADCs/DACs	5
2.2.6	Weitere Peripherien	5
2.3	Routing und Design	5
2.3.1	Footprints	5
2.3.2	Design	6
2.3.3	Fertigung	6
2.3.4	Inbetriebnahme und Probleme	7
2.4	Fazit	8
2.5	Testhardware	9
3	SPI- und SD-Karten-Treiber	9
3.1	Embedded C++20	9
3.2	SPI	10
3.2.1	Anschlussbelegung	10
3.2.2	SPI-Peripherie des STM32G431	11
3.3	Initialisierung der SD-Karte in den SPI-Modus	12
3.4	Lese- und Schreiboperationen	12
4	Dateisysteme	12
4.1	Einleitung	12
4.2	Layout eines Dateisystem	13
4.3	FAT-Dateisysteme	14
4.4	FatFS	15
4.4.1	Einleitung	15
4.4.2	Implementierung	15
5	LittleFS	17
	Quellenverzeichnis	17

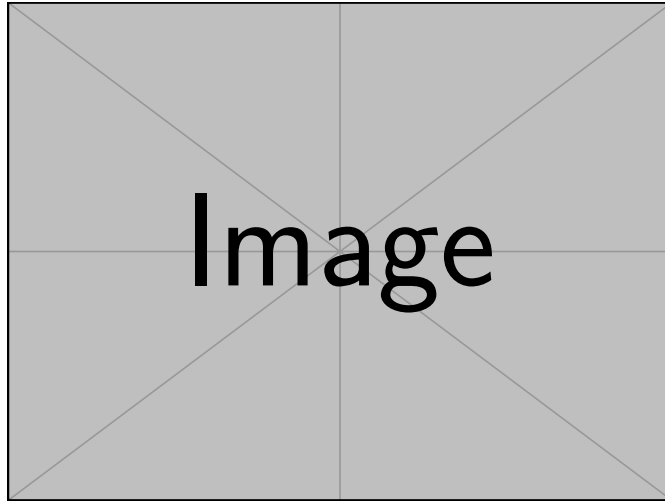


Abbildung 1: Loram Ipsum und so weiter

1 Einleitung

Blah blub

Die gesamte Bachelorarbeit befindet sich öffentlich auf Github unter <https://github.com/miko8278/stm32-SPI-SD-card>.

2 Hardware-Design und Schnittstellen

2.1 Einleitung

Für diese Bachelorarbeit wurde eigens eine PCB in KiCad10 entwickelt. Die vollständigen Dateien, inklusive der Gerberdateien, liegen in dem [Projektverzeichnis auf Github](#).

Das Kernelement der Platine bildet der [STM32G431](#), ein 2019 veröffentlichter Mikrocontroller von STMicroelectronics[2]. Dieser ist preiswert und enthält alle wesentlichen Elemente, die für diese Arbeit benötigt wurden: 128 KB Flash, 32 KB SRAM, 3x SPI-Interfaces und außerdem frei konfigurierbares Direct Memory Access (DMA), das sogenannte DMAMUX. Bei den älteren STM32 waren die DMA-Kanäle festgelegt. Das hatte den Nachteil, dass man schon im Schaltplan sehr darauf achten musste, welche Pins für welche Peripherien verwendet werden, da ansonsten der DMA nicht verwendet werden konnte oder sich mit anderen Peripherien gedoppelt hat.

Weiterhin bietet der STM32G431 2x 12-bit ADCs, 2x 12-bit DACs und zahlreiche Timer, sodass potentielle Anwendungen wie das Einlesen von Sensordaten oder das Ausgeben von Audiodaten umgesetzt werden können.

Eine Übersicht der gesamten Features findet sich im [offiziellen Datenblatt](#), wobei sehr genau gelesen werden muss: Beispielsweise besitzt der Controller zwar 4 DAC Kanäle, aber nur 2 sind gleichzeitig nutzbar. ebenfalls kann eine ADC Genauigkeit von 16 Bit zwar erreicht werden, aber nur in einem speziellen Oversamplingmodus, wodurch die mögliche Abtastrate drastisch sinkt. Als Gehäuseform wurde LQFP48 gewählt, da dieses von Hand lötbar ist und genug Pins für alle benötigten Zwecke herausführt.

Es ist an dieser Stelle erwähnenswert, dass ST Microelectronics im höherwertigen Segment ebenfalls Mikrocontroller anbietet, die eine dezidierte SDMMC-Schnittstelle für den nativen SD-Modus von SD-Karten haben. Mit diesem lassen sich höhere Übertragungsraten erreichen, was für die vorliegende Arbeit aber nicht relevant war.

Die Platine wurde bei dem großen chinesischen Unternehmen [JLCPCB](#) bestellt. Die Bauteile wurden bei dessen Tochterunternehmen [LCSC](#) bestellt. Der Bill of Material (BOM) für die Bauteile befindet sich ebenfalls im Projektverzeichnis.

2.2 Idee und Schaltplan

Die Idee war der Entwurf einer Prototypplatine für einen STM32G431, die mit zwei MicroSD-Kartenslots ausgestattet wird. Darüber hinaus sollte die PCB so simpel wie möglich gehalten werden, da dies mein erster eigener Platinenentwurf ist und die nötige Erfahrung für komplexe Wechselwirkungen und Probleme fehlt.

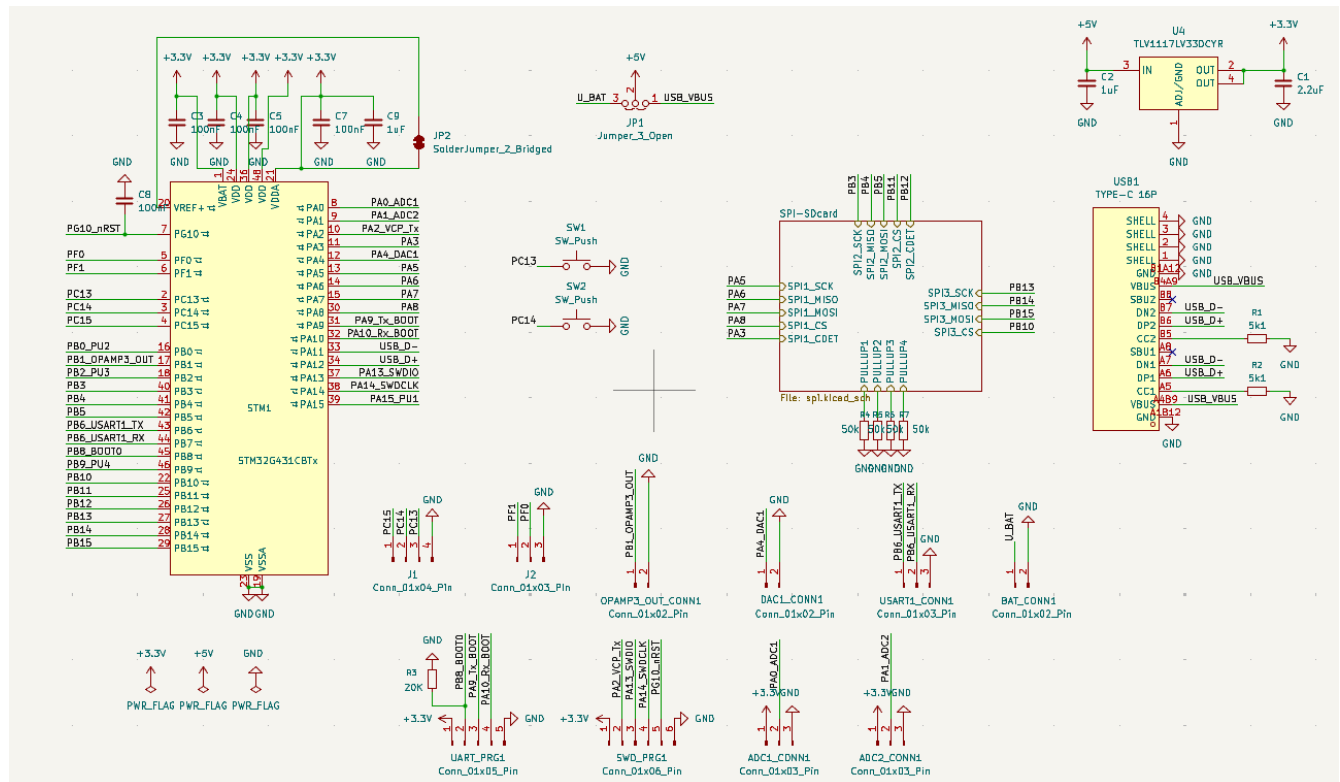


Abbildung 2: Hauptblatt des Schaltplans

Abbildung 2 zeigt den hauptsächlichen Schaltplan. In der Kurzschreibweise von Schaltungen werden oft die Leitungen für GND und VCC/VDD weggelassen. In ähnlichem Stile wurden in dem vorliegenden Schaltplan fast alle anderen Leitungen auch weggelassen und die Pins der Übersichtlichkeit halber mit Labels verbunden. Auch bei einem minimalen Design mussten einige Dinge für eine korrekte Funktionsweise beachtet werden, andere Features wurden hinzugefügt, um eine bessere Auswertung und Nutzung der Platine zu gewährleisten:

2.2.1 Spannungsversorgung

Die Spannungsversorgung soll hauptsächlich über ein 5V USB-C-Netzteil gewährleistet werden. Ein umsteckbarer Jumper ermöglicht allerdings auch die Versorgung über einen externen Akku, sofern er minimal 3,8 V bis maximal 6 V zur Verfügung stellt. Für die Spannungsregelung ist ein **Texas Instruments TLV1117LV33DCYR** Linearregler zuständig, da dieser außer zwei Kondensatoren keine komplexe, externe Beschaltung benötigt. Weiterhin wurden für eine stabile Spannungsversorgung alle VDD-Pins des STM32G431 mit einem 100 nF Kondensator ausgestattet.

2.2.2 USB-C

Die Platine ist mit einem 16-Pin USB Typ C Stecker ausgestattet. Da der Stecker beidseitig in die Buchse gesteckt werden kann, ist jeder Pin in zweifacher Ausführung vorhanden. Hauptsächlich ist der Stecker zur Spannungsversorgung gedacht gewesen. Damit die USB-Spannung von 5 V von einem Netzteil oder anderem Gerät überhaupt bereit gestellt werden kann, müssen die Pins CC1 und CC2 über einen 5,1 k Ω Widerstand auf Masse gezogen werden. Es stehen mit USB-C auch höhere Spannungen zur Verfügung, wenn mit der Spannungsquelle über das USB Power Delivery Protokoll kommuniziert wird. Dies war für die vorliegende Arbeit aber nicht relevant.

Da STM32G431 aber einen USB 2.0 Controller besitzt, wurden ebenfalls die differenziellen Datenpins angeschlossen. Das erweitert die potentiellen Möglichkeiten der Platine für einen Datenaustausch mit dem Computer.

2.2.3 SPI-Interfaces

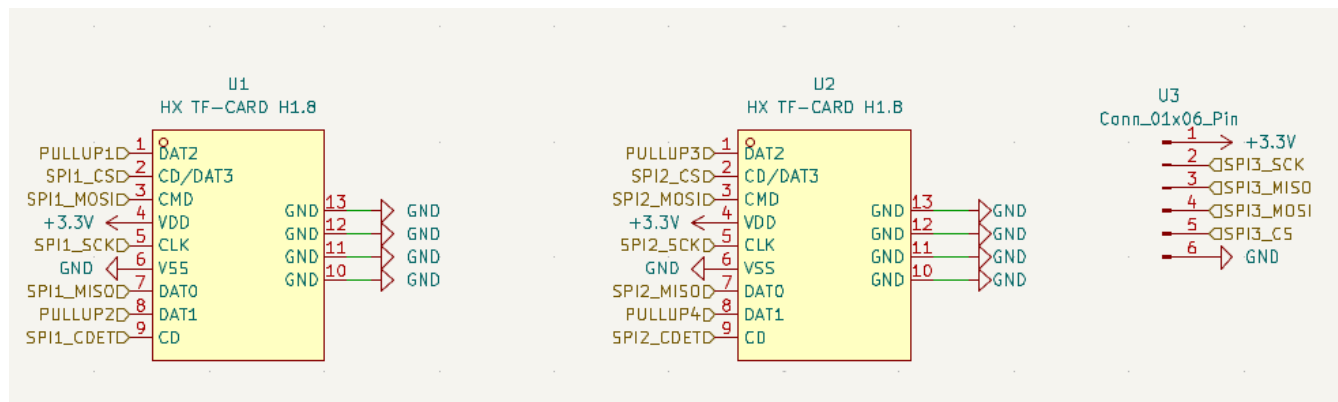


Abbildung 3: Schaltplan für die SPI-Interfaces

Die Hauptfunktionalität der Platine ist die Ansteuerung einer MicroSD Karte über das SPI-Protokoll. Der STM32G431 bietet hierfür eigene SPI-Hardwarecontroller, die als Master fungieren können und einen konfigurierbaren Takt vorgeben. Die Platine verbindet zwei der drei SPI-Peripherien mit MicroSD-Karten Steckplätzen und führt den dritten mit einer Pinleiste aus. Der gewöhnliche Kommunikationsmodus von SD-Karten, der SD-Modus, benötigt zwei weitere Pins. Diese werden per Pull-Up Widerstand auf 3,3 V Spannungspegel gebracht, um ein Übersprechen von den anderen Leitungen auf diese Pins zu unterbinden und damit potentielle Fehler zu minimieren. Die Pull-Up Widerstände sollten zunächst direkt über unge-

nutzte STM32G431 Pins per Software konfiguriert werden. Beim Routing hat sich dies jedoch als schwierig herausgestellt, sodass physische Pull-Up Widerstände verwendet werden sollten. Durch die Änderungen in letzter Minute hat sich hierbei allerdings ein Fehler eingeschlichen und die Widerstände sind zu Pull-Down Widerständen geschaltet worden. Das dritte SPI-Interface wurde per Pinleiste rausgeführt, sodass die Flexibilität gewährleistet ist ein externes SPI-Gerät anzuschließen (wie z.B einen kleinen LCD-Bildschirm), falls dies gewünscht ist.

2.2.4 Programmier-Interfaces

Das Programmierinterface wurde in zwei Versionen herausgeführt:

Einerseits das UART-Programmierinterface über eine 5-Pin Leiste. Dieses erlaubt den STM32 über einen gewöhnlichen UART-Adapter zu programmieren. Der Nachteil ist hierbei allerdings, dass kein Debug-Interface vorhanden ist, sodass die Entwicklung eines Programms erschwert wird.

Andererseits wurden die Pins für das SWD-Interface mit einer 6-Pin Leiste herausgeführt. Dieses Interface benötigt einen dezidierten Programmieradapter. Allerdings lassen sich mit einigen Jumperumstellungen auch die STM-Nucleo Boards dafür nutzen, da diese ebenfalls SWD intern für ihre Programmierung verwenden. ****HIER GENAUER SWD PROG ERKLAEREN MIT PINS UND BILDERN****

2.2.5 VREF und ADCs/DACs

Für die Referenzspannung für die ADC und DAC muss der Pin VREF mit VDDA verbunden werden und mit 1 μ F und 100 nF Kondensatoren abgesichert werden. Alternativ könnte man für eine Referenz den internen VREFBUF nutzen. In der vorliegenden Konfiguration besteht allerdings maximale Flexibilität. Um die Komplexität der Schaltung niedrig zu halten, wurden in diesem Platinendesign keine separaten Versorgungen für die digitale und analoge Spannung verwendet, sodass mit einem höheren Grundrauschen bei DAC und ADC gerechnet werden muss. Es wurden 2x ADCs und 2x DACs über 2-Pin Leisten herausgeführt. Einer der DAC ist intern mit dem OPAMP3 verschaltet und somit gepuffert.

2.2.6 Weitere Peripherien

Für potentielle Debugging und Kommunikationszwecke wurde ebenfalls per 3-Pin Leiste ein USART-Interface herausgeführt. Für eine einfache Eingabemöglichkeit wurden zwei Pins über einen Taster an GND angeschlossen. Diese sollen mit in Software zugeschalteten Pull-Up Widerständen verwendet werden. Weiterhin wurden 5 weitere ungenutzte Pins herausgeführt.

2.3 Routing und Design

2.3.1 Footprints

Da die Platine von Hand gelötet werden sollte, wurden alle Kondensatoren und Widerstände in der 0805 Größe gewählt. Alle Bauteile wurden schon in der Schaltplanphase darauf begutachtet, ob es sie in einer von Hand lötbaren Gehäuseform gibt.

Die Pads praktisch aller Bauteile wurden manuell etwas verlängert, um eine gute Handlötbarkeit zu gewährleisten. LCSC stellt für alle bei Ihnen verfügbaren Bauteile auch Footprints zur Verfügung. Die Footprints

der spezielleren Bauteile, wie die des MicroSD-Kartenslots oder des TI Linearreglers, wurden mit dem Tool easyeda2kicad in einen KiCad Footprint überführt. Diese wiederum mussten teilweise auch leicht angepasst werden, weil z.B der Abstand der Pads zu den Bohrlöchern nicht die für die Produktion zulässigen Maße besessen hatte.

2.3.2 Design

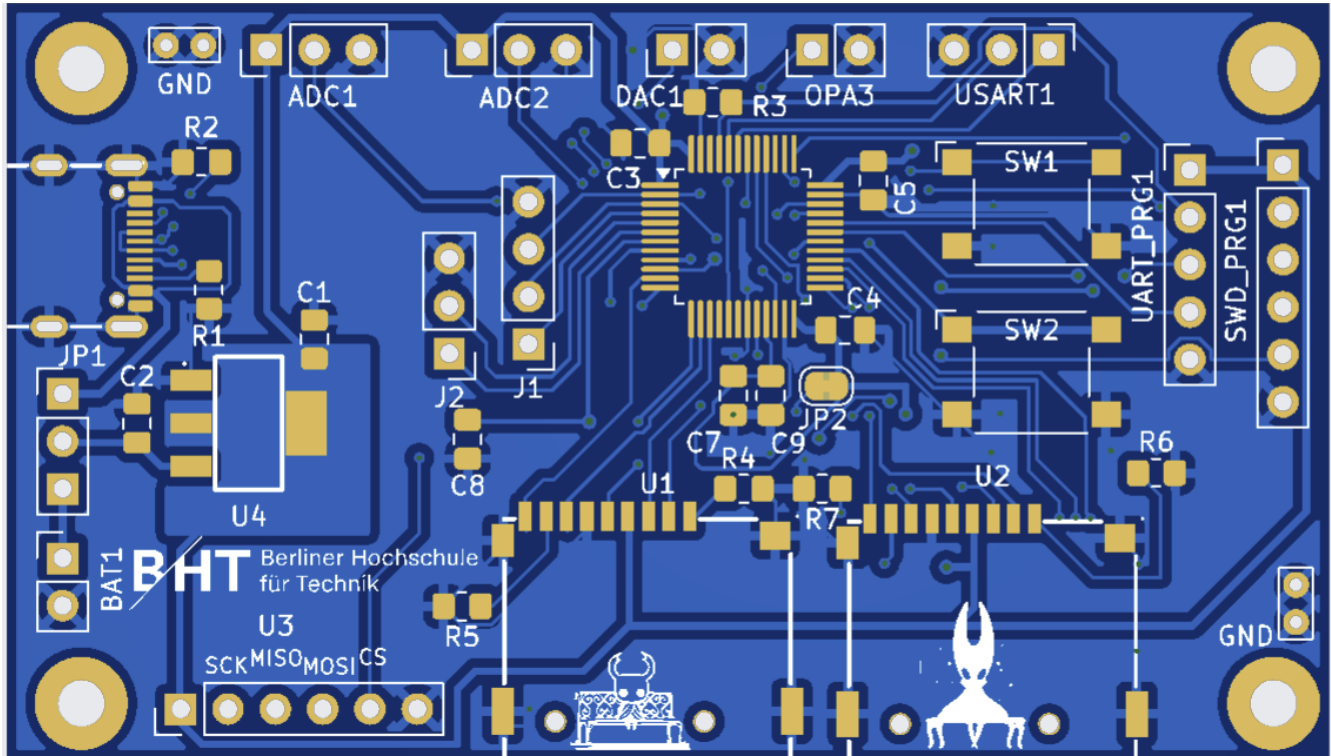


Abbildung 4: Vorschau der entwickelten PCB

Abbildung 4 zeigt die Vorderseite der Platine in dem Zustand, in dem sie bestellt wurde. Die Platine ist zweilagig angelegt und wird nur auf der Vorderseite per Hand bestückt.

Für Signalleitungen wurden 0,2 mm dicke Kupferbahnen, für die stromführenden Leitungen wurde eine Breite von 0,4 mm gewählt. Der grundlegende Abstand von Leiterbahn zu Leiterbahn beträgt 0,2 mm, während der Abstand zu nicht kontaktierten Bohrlöchern 0,25 mm beträgt. Dies sind überwiegend Vorgaben des Produzenten JLCPCB. Die Größe der Platine beträgt 71 mm x 40,5 mm.

Bei zukünftigen Projekten wird für die Beschaltung das echte physische Layout der Pins im Gehäuse stärker beachtet werden. So liegen beispielsweise die Pins PA8 bis PA12 am rechten oberen Rand des LQFP48-Gehäuses, während die Pins PA0 bis PA4 in der linken unteren Ecke liegen. Dies ist beim Schaltungslayout irrelevant, beim Routing der echten Leitungen hingegen führt dies zu überkreuzenden Leitungen. Daher sollte schon in der Schaltungsphase das spätere physische Layout mitbedacht werden.

2.3.3 Fertigung

Die bisherigen Maße und Überlegungen mussten schon konzeptionell und beim Entwurf im CAD-Programm beachtet werden. Für die Fertigung bei JLCPCB mussten weitere Optionen festgelegt werden: Der Ma-

terialtyp ist das übliche FR4 TG135 in der Farbe Blau. Die Löt pads werden nach dem Kupferätzprozess im "LeadFree HASL" Verfahren mit einer Schutzschicht Zinn überzogen. Auf diese Option muss besonders geachtet werden, da in Asien weiterhin bleihaltige Verzinnung üblich ist. Dies ist notwendig, da blankes Kupfer innerhalb kurzer Zeit oxidieren würde. Die Kupferdicke liegt bei den üblichen 35 μm .

Alle weiteren Möglichkeiten sind Spezialoptionen für Sonderanwendungen wie beispielsweise galvanisierte Kanten und sind für diese Platine nicht relevant.

2.3.4 Inbetriebnahme und Probleme

Neben dem bereits erwähnten Problem der falsch beschalteten Pull-Up Widerstände wurden ebenfalls die Pins für die SPI2 und SPI3 Peripherien vertauscht. Dieser Fehler lässt sich durch eine Umkonfigurierung softwareseitig aber lösen.

Der mit Abstand unintuitivste und am schwierigsten auszumachende Fehler der Platine hängt allerdings mit der Spannungsversorgung für die SD-Kartenslots zusammen. Während den Testungen hat sich insbesondere der SD-Kartenslot, der an der SPI3-Peripherie angeschlossen ist, als besonders unzuverlässig herausgestellt. Dieser Fehler ist für den SPI3 SD-Kartenslot fatal und lässt sich ohne ein neues Design kaum beheben.

/figfig:misodip zeigt den Kanal MISO, also die Antwort der SD-Karte auf eine Initialisierungsanfrage. Die erwartete Antwort wäre 0xFF, also ein Signal, das durchgehend 3,3 V ist. Tatsächlich lässt sich aber eine kleine Kondensatorentladungskurve messen. Diese Entladungskurve war noch größer, als noch keine zusätzlichen Kondensatoren provisorisch in der Nähe der SD-Kartenadaptersversorgung und Masse gelötet wurden. Aber auch diese zusätzlichen Kondensatoren befinden sich zu weit weg vom Pin, um den Einbruch gänzlich zu verhindern. Tatsächlich hängt der Spike auch vom verwendeten Netzteil ab, auch wenn sich an der Spannungsversorgung des Netzteils, sowie am TI Linearregler kein nennenswerter Einbruch messen lässt. Ohne einen 10 μF Kondensator nah am SD-Kartenslot ist dieser leider nicht benutzbar.

Die Peripherie an SPI1 hingegen macht weniger Probleme, da sie sich physisch näher an der Spannungsversorgung befindet.

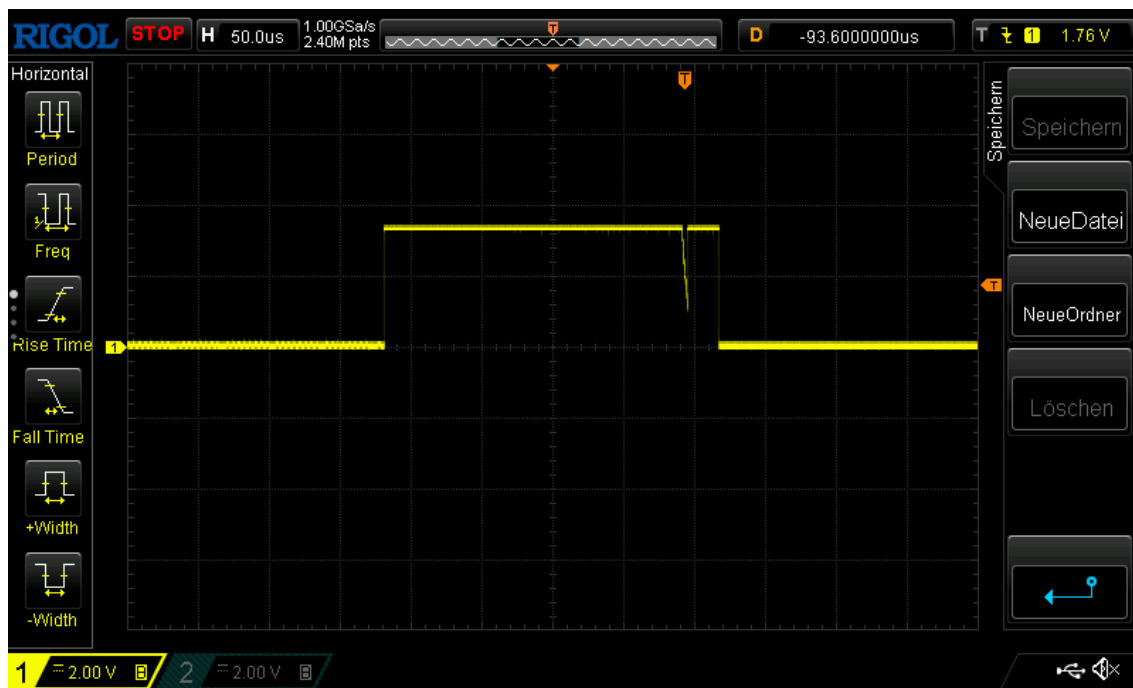


Abbildung 5: MISO: SD-Kartenantwort auf CMD0

Ein schwer aufzufindbarer Fehler, aber leicht lösbarer Fehler, hing mit der Teilbestückung der Platine zusammen. Für erste Tests wurde die Platine nicht vollständig mit allem bestückt. So wurde der Widerstand R3 zunächst vergessen zu löten. *Abbildung 2* macht deutlich, dass dies allerdings ein Pull-Down Widerstand für den Boot-Pin PB8 ist. Wenn sich PB8 auf HIGH befindet, so lädt der STM32 in seinen internen Bootloader, um über die UART-Schnittstelle programmierbar zu sein. Wenn der Pin allerdings unverbunden keinen definierten Pegel erhält, so lädt er mal in den Bootloader und mal nicht, sodass sich die Platine in einem schwer diagnostizierbaren Zustand befindet.

2.4 Fazit

Alles in allem ist das Design für einen ersten Prototypen sehr gelungen und funktional. Der STM32 wird stabil mit Spannung versorgt und lässt sich über ein speziell konfiguriertes Nucleo-Board debuggen und programmieren. Die SPI1- und mit Einschränkungen auch die SPI3-Peripherien lassen sich für die weitere Arbeit verwenden. Eine große Lektion ist, dass schon beim Schaltungsdesign das spätere Layout mitbedacht werden muss, um vorrausschauene Entscheidungen bezüglich der Pinwahl treffen zu können.

SD-Karten scheinen durch ihren internen Aufbau phasenweise sehr kurzfristige Stromspitzen zu verursachen, die der verwendete LDO-Regler nicht ausregeln kann, wenn sich das Bauteil zu weit weg befindet. Daher ist es günstig diese in der Nähe der Spannungsversorgung zu platzieren und mit dezidierten Kondensatoren auszustatten.

Ein großes Ärgernis bei der Fehlersuche, insbesondere im Hinblick auf die SPI-Peripherie, die die Grundlage des SD-Kartentreibers bildet, ist eine fehlende Pinleiste für das Debugging. Ohne diese war es nur mit größter Mühe und Präzision möglich einzelne SPI-Pins mit dem Oszilloskop zu messen. Ein Anschluss an einen Logikanalyser wäre hingegen gar nicht möglich gewesen.

All diese Punkte wären in einer zweiten Version der Platine aber leicht nachzubessern.

2.5 Testhardware

Bestellungen aus China benötigen mehrere Wochen Zeit.

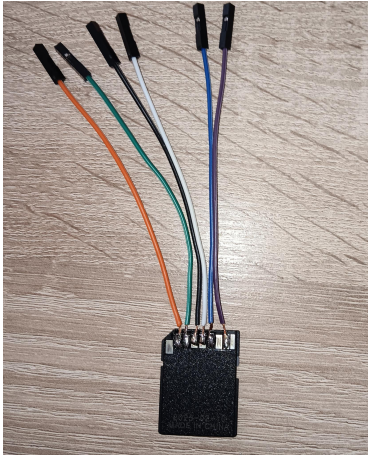


Abbildung 6: Erster gelöteter SD-Adapter

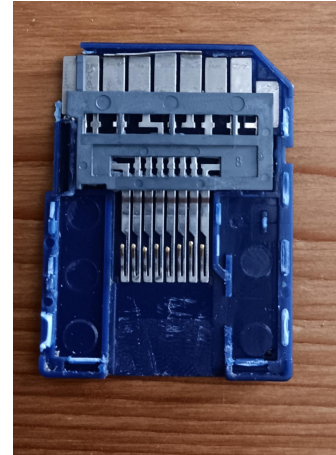


Abbildung 7: Aufgebrochener SD-Adapter

Um in der Zwischenzeit an der Arbeitsstellung weiterarbeiten zu können, wurde der in *Abbildung 6* gezeigte Adapter verwendet in Kombination mit einem ST Nucleo-G431RB. Diese Adapter finden sich meist bei jeder gekauften MicroSD-Karte und haben den Zweck die kleine Karte in ein größeres Format zu überführen. Wie so oft sind improvisierte Lösungen aber nicht sehr zuverlässig: Schon nach zwei Wochen musste ein neuer Adapter, diesmal mit gecrimpten Aderendhülsen, gelötet werden, da sich einzelne Stränge des Litzenkabels gelöst haben und zu Kurzschlüssen geführt haben.

3 SPI- und SD-Karten-Treiber

3.1 Embedded C++20

Für die Entwicklung des SPI- und SD-Karten-Treibers wurde bewusst auf modernes C++20 zurückgegriffen. Die Sprache erlaubt Hardware-Abstraktionen mit Templates und Berechnungen zur Compilezeit mit `constexpr` vollkommen ohne Laufzeitoverhead, sogenannte "zero-cost abstractions". Moderne Sprachfeatures wie `enum class` und `namespace` bieten Typensicherheit und erhöhen die Lesbarkeit. Bedingte Kompilierung mit `if constexpr` wählt nur relevante Zweige zur Compilezeit aus und kompiliert den Rest erst gar nicht. Gleichzeitig besteht Abwärtskompatibilität zu C-Code, sodass beispielsweise die in C89 geschriebene FatFS-Bibliothek problemlos implementiert werden kann.

3.2 SPI

3.2.1 Anschlussbelegung

Tabelle 1: Anschlussbelegung der Standard SD-Karte am STM32G431 im SPI-Modus für SPI1

Pin	SD-Modus	SPI-Modus	Beschreibung (SPI)	STM32 G431 Pin
1	DAT3 / CD	CS	Chip Select (Low-aktiv)	GPIO (z.B. PA8)
2	CMD	DI	Data In (Dateneingang)	PA7 (MOSI)
3	VSS1	VSS	Masse	GND
4	VDD	VDD	Stromversorgung (3,3 V)	3,3 V
5	CLK	SCLK	Taktleitung	PA5 (SCK)
6	VSS2	VSS	Masse	GND
7	DAT0	DO	Data Out (Datenausgang)	PA6 (MISO)
8	DAT1	<i>Nicht genutzt</i>	Pull-Up/nicht belegt	–
9	DAT2	<i>Nicht genutzt</i>	Pull-Up/nicht belegt	–

Tabelle 1 verdeutlicht die Anschlussbelegungen, die überwiegend für die Testhardware am Nucleo-G431RB verwendet wurden.

Abbildung 7 verdeutlicht, dass SD-Adapter lediglich die Leitungen der MicroSD-Karte umlenken. Die neue Belegung für SPI1, mit der die Platine arbeitet, findet sich in *Tabelle 2*.

Tabelle 2: Anschlussbelegung der MicroSD-Karte am STM32G431 im SPI-Modus für SPI1

MicroSD-Pin	Signalname	Verbindung am STM32G431 / Platine
Pin 1	DAT2	3.3V über internen Pull-up
Pin 2	CS	GPIO (z.B. PA8)
Pin 3	DI (MOSI)	PA7
Pin 4	VDD	3.3V Hauptversorgung
Pin 5	SCK	PA5
Pin 6	VSS	GND
Pin 7	DO (MISO)	PA6
Pin 8	DAT1	3.3V über internen Pull-up

3.2.2 SPI-Peripherie des STM32G431

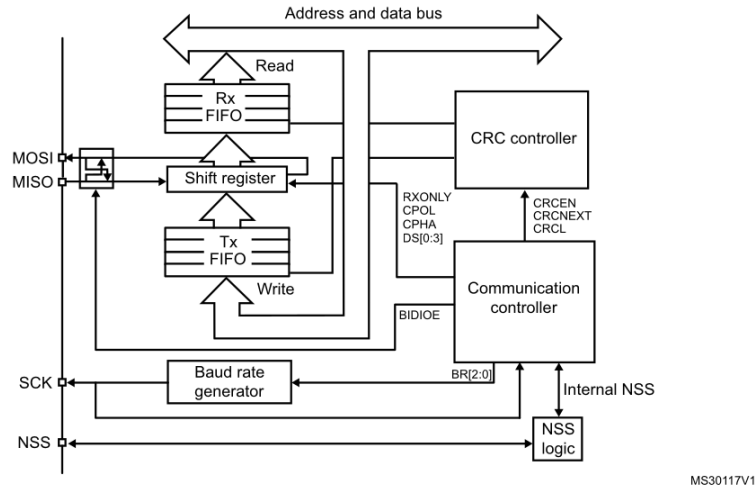


Abbildung 8: Blockdiagramm für die SPI-Peripherie des STMG431[4]

Abbildung 8 zeigt das Blockdiagramm für die SPI-Peripherie der STMG4-Serie. Der relevanteste Teil ist das Shiftregister. Das oberste Bit des Shift-Registers wird auf MOSI ausgegeben. Gleichzeitig wird das Bit auf MISO eingelesen. Es liegt immer eine Full-Duplex Übertragung vor. Möchte man ein Byte empfangen, so muss immer auch eines gesendet werden. Aus dem Rx-FIFO können Daten gelesen werden, sobald das RXP-Flag gesetzt ist. Dieses Flag wird gesetzt, wenn so viele Bits wie im Feld "Data size" des SPIx_CR2 Registers mit Daten gefüllt wurden.

Der Baudratencontroller ist für die SCLK und somit für die Übertragungsgeschwindigkeit zuständig. Er nutzt den APB-Bus-Takt und teilt ihn, falls im BR-Register konfiguriert, durch einen Prescaler.

Für den Zweck der SD-Karten-Kommunikation wurde die SPI-Peripherie als Master konfiguriert mit einem frei wählbaren, softwareseitig zuschaltbaren NSS-Pin (chip select).

```

1  template<uintptr_t PortBase, uint32_t Pin>
2  struct ChipSelect {
3  private:
4      static inline GPIO_TypeDef* const port = reinterpret_cast<GPIO_TypeDef*>(PortBase);
5  public:
6      ChipSelect() {
7          port->BSRR = (1U << (Pin + 16));
8      }
9
10     ~ChipSelect() {
11         port->BSRR = (1U << Pin);
12     }
13 };

```

Quellcode 1: Chip select als Klasse

Der Quellcode 1 zeigt die Stärke von C++ im Gegensatz zu C in diesem Kontext. Der NSS-Pin muss am An-

fang der meisten SD-Funktionen wie `SD_ReadBlock(...)` oder `SD_SendCmd(...)` auf Low konfiguriert werden und beim Verlassen der Funktion wieder auf High. Mit `ChipSelect<GPIOA_BASE, 8> chipselect_tmp;` würde ein Objekt erzeugt werden. Beim Konstruieren, also dem Erzeugen des Objektes, würde der Konstruktor Pin 8 von GPIOA auf Low setzen und beim Verlassen der Funktion würde der Destruktor aufgerufen werden und den Pin wieder auf High setzen. In C müssten manuell alle Stellen gefunden werden, bei dem die Funktion verlassen wird oder Laufzeitkosten in Kauf genommen werden mit dem Aufruf einer Helferfunktion. Die Informationen welcher GPIO-Port mit welchem Pin benutzt werden soll steht zur Compilezeit fest, der verwendete Cast ist `static inline const`, sodass ein guter C++-Compiler die gesamte Klasse vollständig auf zwei direkte Registerzugriffe reduzieren kann.

3.3 Initialisierung der SD-Karte in den SPI-Modus

3.4 Lese- und Schreiboperationen

Ein Sektor bezeichnet die kleinste physikalisch adressierbare Speichereinheit eines Datenträgers. Ein Block bezeichnet dagegen die kleinste logische Speichereinheit, die von einem Betriebssystem oder Dateisystem adressiert wird. Im Zusammenhang von Flashspeicher wird die Page als kleinste programmierbare Einheit beschrieben, wohingegen der Erase Block die kleinste löschbare Einheit beschreibt. Leider werden in Spezifikationen und Literatur die Begrifflichkeiten nicht eindeutig verwendet. So spricht man im Zusammenhang von SD-Karten meist von Blöcken, die beschrieben werden, obwohl der Begriff des Sektors hier manchmal synonym verwendet wird.

4 Dateisysteme

4.1 Einleitung

Eine umfassende Einführung in die Grundlagen von Dateisystemen wird von Tanenbaum und Bos in ihrem Standardwerk *Modern Operating Systems* bereitgestellt [1, S. 263 ff.]. Laut den Autoren werden drei essenzielle Anforderungen an die Langzeitspeicherung von Daten gestellt:

1. Es muss möglich sein, eine sehr große Menge an Informationen zu speichern.
2. Die Informationen müssen das Beenden des Prozesses, der sie verwendet, überdauern.
3. Mehrere Prozesse müssen gleichzeitig auf die Informationen zugreifen können.

Tanenbaum und Bos nutzen in ihrem Buch den Begriff des Prozesses zur Ressourcenverwaltung und -abstraktion, während diese Notwendigkeit in der vorliegenden Arbeit entfällt, da das Programm direkte Kontrolle über die gesamte Hardware des Mikrocontrollers besitzt. Im Prinzip könnte das Problem eine große Menge an Information zu speichern bereits mit zwei Operationen lösen:

1. Einen wählbaren Block lesen
2. Einen wählbaren Block schreiben

Das sind exakt die Operationen, die in dem vorherigen Kapitel 3 für die SD-Karte implementiert wurden. In der Praxis sind diese zwei Operationen allerdings zu primitiv, um Daten effektiv zu verwalten. Tanenbaum und Bos stellen beispielhaft folgende Fragen:

- Wie findet man eine bestimmte Information
- Wie verhindert man (auf Mehrbenutzersystemen), dass ein Benutzer nicht die Daten eines anderen ausliest?
- Woher weiß man, welche Blöcke frei sind?

Für diese und viele andere praktische Probleme hilft das Konzept der Datei. Die Datei ist etymologisch ein Neologismus aus Daten und Kartei. Im Kontext moderner Betriebssysteme definieren Tanenbaum und Bos[1] Dateien als logische Einheiten von Information, die von Prozessen erzeugt werden und diese überdauern. Eine Datei soll erst gelöscht werden, wenn ein Besitzer dies explizit veranlasst.

Wie Dateien strukturiert, benannt, aufgerufen, genutzt, geschützt, implementiert und verwaltet werden ist Sache des Dateisystems. Ein Dateisystem löst (je nach Funktionsumfang) viele der oben gestellten Anforderungen. Aus Sicht des Nutzers ist der wichtigste Aspekt eines Dateisystems, wie es sich darstellt: Was eine Datei ausmacht, wie Dateien benannt und geschützt werden, welche Operationen auf Dateien erlaubt sind. Die Details darüber, ob verkettete Listen oder Bitmaps verwendet werden, um den freien Speicherplatz zu verwalten, oder wie viele Sektoren ein logischer Festplattenblock enthält, sind für den Nutzer von keinem Interesse, obwohl sie für die Entwickler des Dateisystems von großer Bedeutung sind[1]. Zusammenfassend lässt sich sagen, dass ein Dateisystem eine für den Anwendungszweck geeignete Struktur in einen rohen Speicherbereich bringt. Während Tanenbaum und Bos den Schwerpunkt auf Dateisystemen für Betriebssysteme legen, was sicherlich der übliche Anwendungsbereich ist, geht es in dieser Arbeit um die Verwendung von Dateisystemen in eingebetteten Systemen, bzw. konkret auf einem STM32-G431 und deren Vor- und Nachteile.

4.2 Layout eines Dateisystem

Während das Design eines Dateisystems grundsätzlich frei entschieden werden kann, folgt die Struktur meist einem ähnlichen Muster, sodass die Klärung einiger üblicher Begriffe nötig ist.

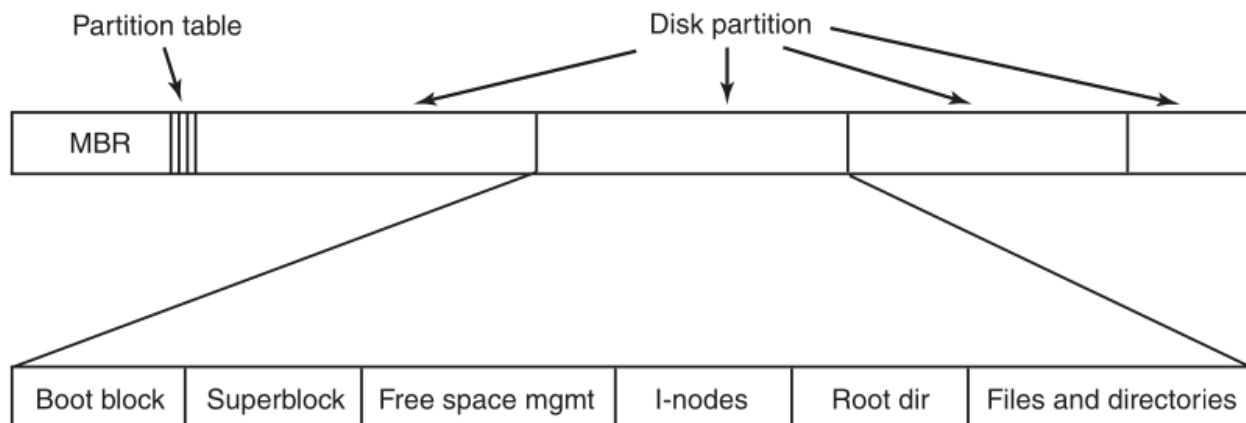


Abbildung 9: Ein mögliches Dateisystemlayout [1, S. 282]

Abbildung 9 zeigt ein mögliches Layout eines Dateisystems nach Tanenbaum und Bos [1, S. 282]. Der erste Sektor eines Speichermediums wird reserviert für den MBR (Master Boot Record) und wird üblicherweise zum Starten eines Computers verwendet. Das Ende des MBR enthält die Partitionstabelle, in welcher eine der Partitionen als aktiv markiert ist. Ein physisches Medium kann mehrere Partitionen und somit mehrere Dateisysteme beinhalten. Wenn der Computer gebootet wird, dann stellt ein Programm fest, welche Partition aktiv ist und führt die Anweisungen in dem "Boot block" aus. Üblicherweise folgt darauf Superblock, in dem wichtige Informationen über das Dateisystem stehen wie der Typ des Dateisystems oder die Anzahl der vorhandenen Blöcke. Im Anschluss folgen typischerweise Informationen über die freien Blöcke des Dateisystems, die beispielsweise in Form einer Zeigerliste organisiert sind. Darauf folgen die sogenannten I-Nodes, ein Array von Datenstrukturen, das für jede Datei existiert und alle relevanten Datei-Metadaten enthält. Danach schließt sich das Wurzelverzeichnis (Root Directory) an, welches die Wurzel der gesamten Dateisystemstruktur bildet. Der verbleibende Speicherplatz des Datenträgers steht schließlich für alle weiteren Verzeichnisse und die eigentlichen Dateien zur Verfügung[1, S. 282].

4.3 FAT-Dateisysteme

Das FAT-Dateisystem wurde 1980 von Microsoft entwickelt und erstmals von MS-DOS unterstützt. Es wurde ursprünglich als einfaches Dateisystem entwickelt, das für Disketten mit einer Größe von weniger als 500 KB geeignet war. FAT ist die Abkürzung für File Allocation Table (Dateizuordnungstabelle). Derzeit gibt es vier FAT-Untertypen: FAT12, FAT16, FAT32 und seit 2006 exFAT als Nachfolger[8]. Im Laufe der Zeit wurden die Spezifikationen erweitert, um mit zunehmender Kapazität auch größere Speichermedien zu unterstützen.

Tabelle 3: Technische Adressierungsgrenzen der FAT-Dateisysteme[9]

Dateisystem	Max. Dateigröße	Max. Mediengröße	Hauptsächlicher zweck	Einsatz-
FAT12	16 MB	16 MB	Klassische 3,5-Zoll-Disketten (1,44 MB)	
FAT16	2 GB	2 GB	MS-DOS-Systeme, MP3-Player, alte Maschinen	frühe CNC-
FAT32	4 GB	2 TB	USB-Sticks, ältere oder Konsolen	Kameras
exFAT	512 TB oder mehr	512 TB oder mehr	Moderne Speicherkarten,	SDXC/SDUC- externe SSDs

Tabelle 3 zeigt die maximal möglichen, *üblichen* Datei- und Mediengrößen für die FAT-Dateisysteme und soll in erster Linie die Entwicklung der Spezifikation und die Probleme der älteren Dateisysteme darstellen. So ist es einem FAT32-Dateisystem beispielsweise nicht möglich eine 12 GB große .iso Datei aufzunehmen. Wie ergeben sich die maximalen Datei und Mediengrößen? Der Unterschied zwischen einem Sektor und

einem Block wurde bereits erläutert: Der Sektor ist die physisch kleinste adressierbare Einheit, während der Block die logisch die kleinste adressierbare Einheit ist. Im Kontext der FAT-Dateisysteme wird der Block allerdings Cluster genannt, während der Begriff des Blocks meist im Unix-Kontext üblich ist. FAT12 stehen 12 Bits, FAT16 16 Bits und FAT32 28 Bits für die Adressierung der Cluster zur Verfügung. Mit größeren Clustergrößen ergibt sich also eine größere Mediengröße. Große Cluster haben allerdings den Nachteil, dass sehr kleine Dateien mindestens so groß werden wie die Clustergröße. Weiterhin wurden Clustergrößen immer gedeckelt, üblicherweise mit 32 KB Maximalgröße für FAT16 und FAT32. Dies führt bei FAT16 zu den genannten 2 GB und bei FAT32 zu 8 TB maximaler Mediengröße. Hinzu kommt allerdings eine weitere Limitierung: Der MBR enthält die Partitionstabelle. Dieser zählt allerdings in 512 Byte großen Sektoren und ihm steht lediglich ein Feld von 32 Bit für die Adressierung zur Verfügung. Folglich kann das FAT32-Dateisystem zwar eine größere Medien unterstützen, wird dann aber von der MBR-Partitionstabelle gebremst[9][8].

Die moderne Lösung dieser Limitierung liegt in der Verwendung des GUID Partition Table (GPT), die ebenfalls viele weitere Vorteile zum MBR bringt wie beispielsweise eine erhöhte Anzahl von Primärpartitionen und ein Backup der Partitionstabelle am Ende des Speichermediums [11].

Die Dateigröße bei FAT32-Systemen ist durch ein 32 Bit-Adressierungsfeld limitiert. Dieses zählt in Bytes, wodurch sich die charakteristischen 4 GB Dateigröße ergeben, die für heutige Zwecke manchmal nicht ausreichend sind[9][8]. Zu diesem Zweck wurde mit exFAT als moderne FAT-Variante 2006 eingeführt.

4.4 FatFS

4.4.1 Einleitung

FatFs ist ein plattformunabhängiges, quelloffenes Software-Modul zur Implementierung von FAT/exFAT-Dateisystemen in ressourcenbeschränkten, eingebetteten Systemen. Es wurde von dem Entwickler ChaN[7] entworfen und zeichnet sich durch einen extrem geringen Speicherbedarf sowohl im Programm- als auch im Arbeitsspeicher aus. Das Modul ist vollständig in ANSI C (C89) geschrieben. Dadurch ist es hardwareunabhängig auf praktisch jedem gängigen Mikrocontroller (z. B. ARM, AVR, PIC oder STM32) ohne Änderungen des Codes direkt lauffähig.

Die Architektur trennt das Dateisystem strikt in zwei logische Schichten: Das Application Interface stellt der Anwendung abstrakte, bekannte Funktionen zur Datei- und Verzeichnisverwaltung wie `f_open`, `f_read`, `f_write` und `f_close` zur Verfügung, während das Media Access Interface (Disk I/O) die Hardware vollkommen vom Dateisystem entkoppelt.

4.4.2 Implementierung

```
1  DSTATUS disk_initialize(BYTE pdrv);
2  DSTATUS disk_status(BYTE pdrv);
3  DRESULT disk_read(...);
4  DRESULT disk_write(...);
5  DRESULT disk_ioctl(...);
```

Quellcode 2: Funktionen, die in der `diskio.cpp` implementiert werden mussten

ffconf.h

Die Datei `ffconf.h` ist die zentrale Konfigurationsdatei des FatFs-Moduls. Über den C-Präprozessor lässt sich der Funktionsumfang auf die Hardware-Ressourcen des Mikrocontrollers zuschneiden.

- **Anpassbarer Funktionsumfang:**

- `FF_FS_READONLY`: Schaltet das Schreiben komplett ab, was Flash-Speicher einspart.
- `FF_FS_MINIMIZE`: Entfernt gezielt ungenutzte APIs wie z.B. die Verzeichnisoperationen (`f_mkdir`, `f_opendir`) oder Suchfunktionen.
- `FF_USE_MKFS`: Aktiviert oder deaktiviert die Formatierungsfunktion `f_mkfs`.
- `FF_USE_FASTSEEK`: Beschleunigt den wahlfreien Zugriff auf große Dateien.
- `FF_USE_FIND`: Aktiviert Suchfunktionen wie `f_findfirst()` und `f_findnext()`.

- **Dateinamen und Zeichenkodierung:**

- `FF_USE_LFN`: Schaltet die Unterstützung für lange Dateinamen frei.
- `FF_CODE_PAGE`: Legt die Codepage fest, um Sonderzeichen korrekt darzustellen.
- `FF_LFN_UNICODE`: Aktiviert Unicode-Unterstützung für Dateinamen.

- **Speicher- und Laufwerksoptionen:**

- `FF_VOLUMES`: Definiert die Anzahl der logischen Laufwerke, die gleichzeitig verwaltet werden können.
- `FF_MAX_SS` & `FF_MIN_SS`: Definiert die vom Dateisystem unterstützten logischen Sektorgrößen. Üblicherweise besitzen SD-Karten eine logische Sektorgröße von 512 Byte, FatFs kann jedoch auch größere Sektorgrößen unterstützen.
- `FF_FS_LOCK`: Legt fest, wie viele Dateien gleichzeitig vor konkurrierenden Zugriffen geschützt werden können.

- **Systemeinstellungen:**

- `FF_FS_TINY`: Reduziert den RAM-Bedarf pro Datei, indem interne Puffer gemeinsam genutzt werden.
- `FF_FS_EXFAT`: Aktiviert die Unterstützung des exFAT-Dateisystems für SDXC-Speicherkarten.
- `FF_FS_NORTC`: Verwendet feste Zeitstempel, wenn keine Echtzeituhr (RTC) vorhanden ist.
- `FF_FS_REENTRANT`: Aktiviert die Thread-Sicherheit des Dateisystems für Mehrthread-Anwendungen.

Darüber hinaus stellt FatFs zahlreiche weitere Konfigurationsoptionen bereit. Dazu gehören beispielsweise Einstellungen für mehrere Partitionen (`FF_MULTI_PARTITION`), Laufwerksbezeichnungen (`FF_STR_VOLUME_ID`), Dateiattribute (`FF_USE_CHMOD`), Laufwerkslabels (`FF_USE_LABEL`) oder die Unterstützung von TRIM-Befehlen für Flash-Speicher (`FF_USE_TRIM`). Dadurch lässt sich das Dateisystem flexibel an die Anforderungen der jeweiligen Embedded-Anwendung anpassen. Eine vollständige Auflistung findet sich einerseits direkt in der `ffconf.h` mit hilfreichen Kommentaren, andererseits auf der [offiziellen Webseite des Entwicklers](#).

Aufgrund der verfügbaren Ressourcen des STM32G431 wurde auf eine weitgehend vollständige FatFs-Konfiguration zurückgegriffen, einschließlich der Unterstützung langer Dateinamen und des exFAT-Dateisystems.

5 LittleFS

Quellenverzeichnis

- [1] Tanenbaum, A. S. & Bos, H. (2015). *Modern Operating Systems* (4. Aufl.). Vrije Universiteit Amsterdam, The Netherlands: Prentice Hall.
- [2] STMicroelectronics. (2026). *STM32G431RB Mainstream Arm Cortex-M4 MCU Product Page*. Offizielle Produkt Information. Abgerufen von <https://www.st.com/en/microcontrollers-microprocessors/stm32g431rb.html> (Besucht am 26. Juli 2026).
- [3] STMicroelectronics. (2021). *STM32G431x6 STM32G431x8 STM32G431xB Datasheet (DS12589 Rev 6)*. Offizielle Spezifikation. Abgerufen von <https://www.st.com/resource/en/datasheet/stm32g431rb.pdf> (Besucht am 26. Juli 2026).
- [4] STMicroelectronics. (2025). *RM0440 Reference manual: STM32G4 series advanced Arm-based 32-bit MCUs (Rev 9)*. Offizielle Spezifikation. Abgerufen von https://www.st.com/resource/en/reference_manual/rm0440-stm32g4-series-advanced-armbased-32bit-mcus-stmicroelectronics.pdf (Besucht am 26. Juli 2026).
- [5] Ababei, C. (2013). *Lecture 12: SPI and SD cards*. Vorlesungsskript, EE-379 Embedded Systems and Applications, University at Buffalo. Abgerufen von https://www.dejazzer.com/ee379/lecture_notes/lec12_sd_card.pdf (Besucht am 6. Juli 2026).
- [6] SD Group Matsushita Electric Industrial Co., Ltd. (Panasonic) (2006). *SD Specifications Part 1: Physical Layer Simplified Specification, Version 2.00*. Technische Spezifikation, SD Card Association. Abgerufen von https://users.ece.utexas.edu/~valvano/EE345M/SD_Physical_Layer_Spec.pdf (Besucht am 8. Juli 2026).
- [7] ChaN (2024). *FatFs - Generic FAT Filesystem Module*. Software-Dokumentation, Electronic Lives Manufacturing. Abgerufen von <https://elm-chan.org/fsw/ff/> (Besucht am 23. Juli 2026).
- [8] ChaN (2020). *FAT Filesystem*. Software-Dokumentation, Electronic Lives Manufacturing. Abgerufen von https://elm-chan.org/docs/fat_e.html (Besucht am 24. Juli 2026).
- [9] Microsoft (2023). *File System Functionality Comparison*. Software-Dokumentation, Microsoft Learn. Abgerufen von <https://learn.microsoft.com/en-us/windows/win32/fileio/filesystem-functionality-comparison> (Besucht am 24. Juli 2026).
- [10] Microchip Technology Inc. (2024). *How to interface a microSD media with Microchip SD/MMC Card Readers*. Support Knowledge Base. Abgerufen von <https://support.microchip.com/s/article/How-to-interface-a-microSD-media-with-Microchip-SD-MMC-Card-Readers> (Besucht am 8. Juli 2026).

- [11] UEFI Forum. (2024). *UEFI Specification Version 2.11 (Chapter 5: GUID Partition Table Format)*. Official Specification. Abgerufen von https://uefi.org/specs/UEFI/2.11/05_GUID_Partition_Table_Format.html (Besucht am 24. Juli 2026).
- [12] Michael Eiler, *C++20: Building a Thread-Pool With Coroutines*
<https://blog.eiler.eu/posts/20210512/>